WORKSHOP · 90 MIN

SQL AI Agents in Production

From notebook demo to a system you can trust in production.

SQL AI Agents in Production · Workshop

ABOUT THIS WORKSHOP

90 minutes, end to end.

For data and ML engineers comfortable with Python and SQL who've called an LLM API at least once. No prior agent experience required.

What you'll leave with

The line between a single LLM call and an orchestrated SQL AI agent. What a production agent's components are, and which lever — context, guardrails, model — moves quality the most.

How we'll spend the time

Three demos against a real repo (`notebook/02`, `app/agent_app.py`, and the eval results). Five concept sections in between. Q&A at the end.

AGENDA

Six topics, one deck.

TOPIC 01

What is a SQL AI Agent?

15 min · demo

TOPIC 02

SQL AI Agent in Production

10 min

TOPIC 03

Context is Everything

12 min · demo

TOPIC 04

SQL AI Agent Guardrails

10 min · demo

TOPIC 05

Model Performance & Evaluation

15 min · demo

TOPIC 06

Best Practices

8 min · recap

Q&A 10 min

TOPIC 01 · 15 MIN

What is a SQL AI Agent?

More than a single LLM call. A pipeline that turns a question into a row of data.

TOPIC 01 · PROMPT VS. AGENT

One LLM call ≠ a SQL AI agent.

✕ Just a prompt

Single LLM call

- Returns text — not rows.
- Doesn't know the schema.
- Doesn't run the SQL.
- If the SQL is broken, nothing tells us.

Useful as a step. Not a system.

✓ SQL AI agent

Orchestrated pipeline

- Builds a prompt that includes the schema.
- Calls the LLM, then parses the response.
- Runs the query against the database.
- Hands rows back to the user.

One LLM call is one step inside the loop.

TOPIC 01 · THE PIPELINE

Seven components, in order.

USER · PIPELINE · DATA

One LLM call is one step inside the loop — not the whole thing.



Same components on every modern SQL agent — names vary, shape doesn't.

TOPIC 01 · WHERE EACH PIECE LIVES

From diagram to code.

COMPONENT	WHERE IT LIVES IN THE REPO
Prompt template	`sql_ai_agent/prompt_handler.py` · `set_prompt_template()`
API handler	`ChatOpenAI` client → Docker Model Runner
LLM API	`gemma4:100k` via `http://model-runner.docker.internal/engines/v1`
Parse the response	`sql_ai_agent/parse_query.py` · `is_markdown_code_chunk` · `extract_code_from_markdown`
DB handler	`sql_ai_agent/db_handler.py` · `get_tbl_attr` · `query_execute` + Ibis connection

TOPIC 01 · DEMO

Let's see it run.

notebook/02_simple_sql_ai_agent.ipynb

Eleven cells. Seven components. One question:

`"How many passengers landed during 2024?"`

Watch for two things: the schema is the agent's only knowledge of the table, and there's no safety net.

```
# the entire agent — three lines once everything is wired
chain = prompt_template | llm
output = chain.invoke({"question": q, "tbl_name": tbl, "schema": schema})
con.sql(output.content).execute()
```

TOPIC 01 · TAKEAWAY

A SQL AI agent is the seven-component pipeline, wired together.

One LLM call is one step inside that loop.

`user question` · `prompt template` · `API handler` · `LLM` · `parse` · `DB handler` · `return data`

Next → Topic 02 · what changes when this moves to production.

TOPIC 02 · 10 MIN

SQL AI Agent in Production.

Same pipeline at the core. New layers around it.

TOPIC 02 · WHAT "PRODUCTION" MEANS

More concerns, not more code.

The seven-component pipeline still works. Production wraps five concerns around it.

Correctness

What happens when the model returns broken SQL?

Safety

What happens when it returns a destructive statement?

Observability

How do we know what happened overnight?

Cost

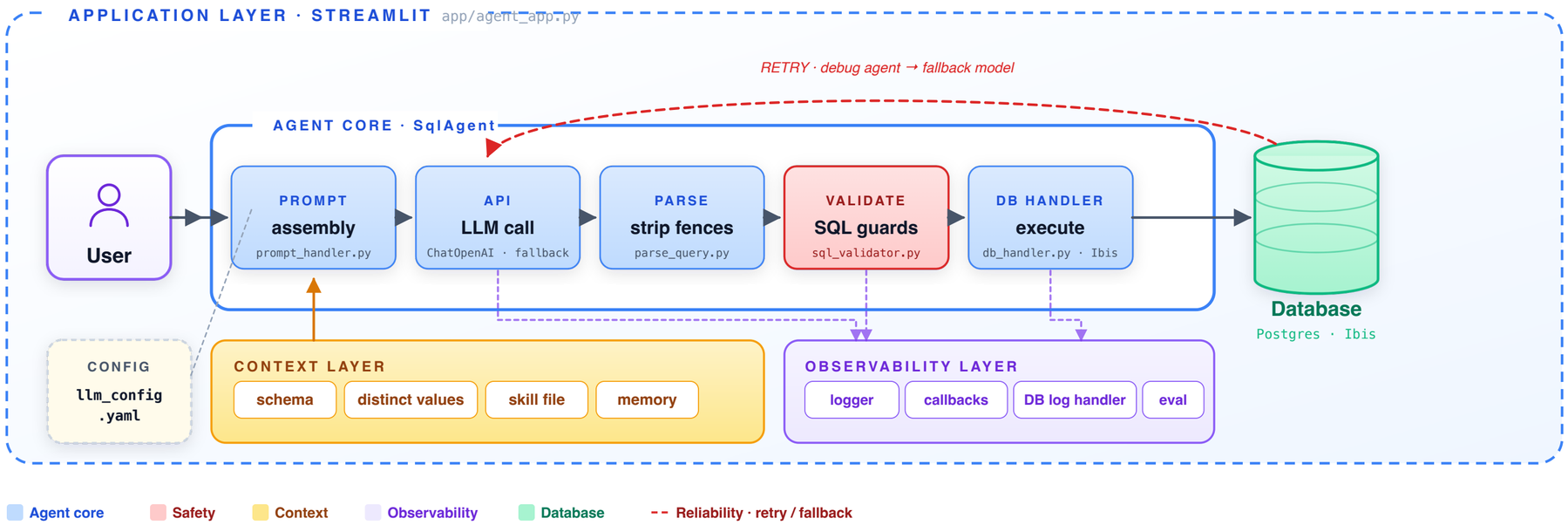
Are we over-spec for the questions our users actually ask?

Evolvability

How hard is it to swap the provider or add a domain?

TOPIC 02 · PRODUCTION ARCHITECTURE

Topic 01's pipeline at the core.



Same seven-component pipeline at the core. Five layers wrap it: application, context, safety, reliability, observability.

TOPIC 02 · THE SIX LAYERS

Where each module lives.

Application

`app/agent_app.py`

Streamlit UI. Renders results, exposes settings.

Agent core

`SqlAgent.py`
`parse_query.py`
`db_handler.py`

Orchestrates prompt → API → parse → DB.

Context

`prompt_handler.py`
`skill_manager.py`
`InMemoryChatMessageHistory`

Schema, distinct values, skill file, memory. Topic 03 opens it.

Safety

`sql_validator.py`

Read-only, multi-statement protection, LIMIT enforcement. Topic 04 opens it.

Reliability

`debug retry loop`
`fallback model`
`llm_config.yaml`

Re-prompt on failure; switch models when the primary fails.

Observability

`logger.py · callbacks.py`
`log_database.py`
`testing/`

Structured logs, token/timing capture, offline eval. Topic 05 opens it.

TOPIC 02 · TAKEAWAY

Production = the seven-component core + safety + reliability + observability.

Six layers, one orchestrator.

`application` · `agent core` · `context` · `safety` · `reliability` · `observability`

Next → Topic 03 · context is everything.

TOPIC 03 · 12 MIN

Context is everything.

The cheapest, biggest lever on agent quality.

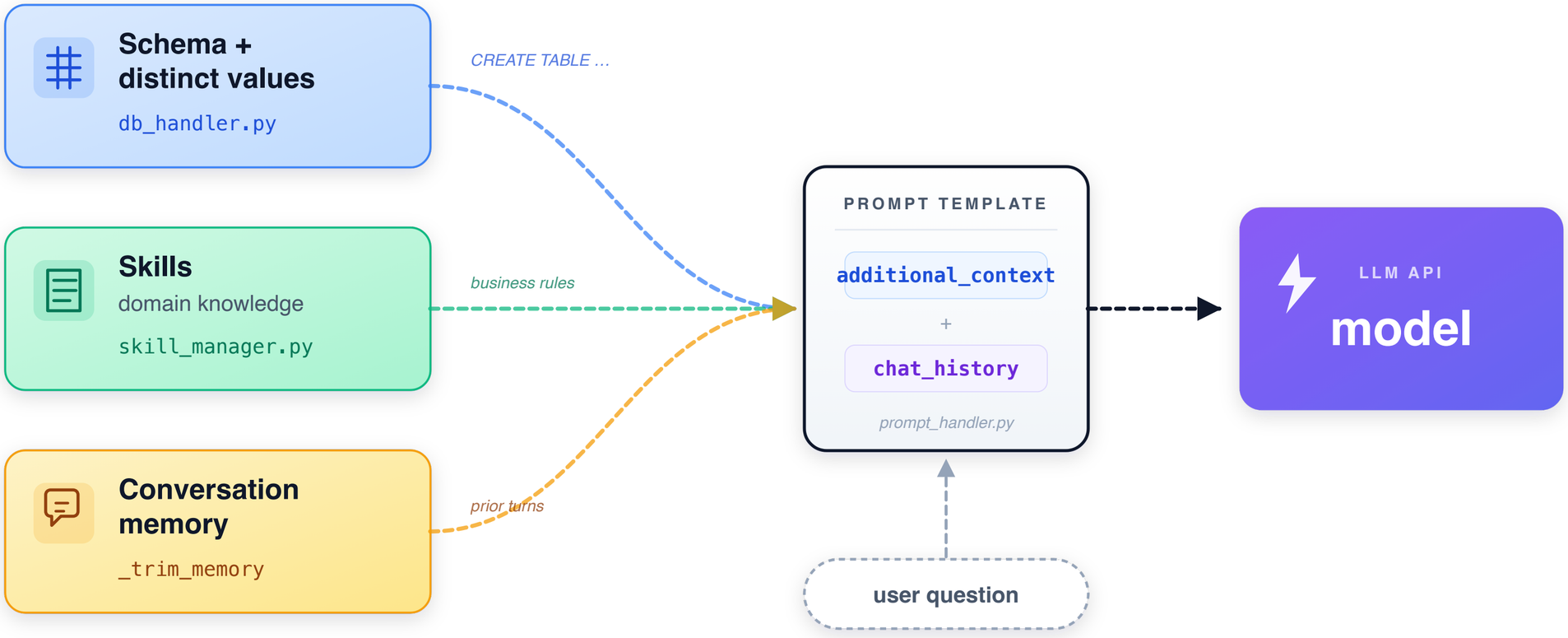
TOPIC 03 · THE CLAIM

The same model with better context beats a bigger model with worse context.

Most agent failures aren't reasoning failures. They're context gaps.

TOPIC 03 · THREE CONTEXT LAYERS

Three layers, one prompt.



An LLM only knows what we put in the prompt. Schema, business rules, prior turns — none of it is implicit.

TOPIC 03 · LAYER 1

Schema + distinct values.

1

What it is

The structural description of the table. Schema is always-on. Distinct values is opt-in: it tells the model that `"Activity Type Code"` holds `'Deplaned'`, `'Enplaned'`, `'Thru / Transit'` — not `'arrival'` or `'departure'`, which is what an LLM will guess otherwise.

How it's wired

`get_tbl_attr` + `get_character_distinct_values` in `sql_ai_agent/db_handler.py`. Formatted by `format_distinct_values_for_prompt` and appended to `additional_context`.

Sidebar · Include Distinct Character Values

TOPIC 03 · LAYER 2

Skills — the domain knowledge file.

2

What it is

A markdown file under `skills/` that encodes what a human analyst knows but a generic LLM doesn't: column meanings (`Deplaned` =arrival), business rules (exclude transit, use Operating Airline), query patterns, percentage/growth math, common pitfalls.

How it's wired

`SkillManager.load_skill(...)` in `sql_ai_agent/skill_manager.py`. `SqlAgent.__init__` tries `<tbl_name>`, `<tbl_name>_context`, `sfo_<tbl_name>_context`. For us: `skills/sfo_air_traffic_context.md` (~250 lines).

Code-only today (no sidebar toggle)

TOPIC 03 · LAYER 3

Conversation memory.

3

What it is

A buffer of prior user/assistant turns. Lets the agent resolve *"what about 2023?"* against the previous question. FIFO; capped at `memory_size` pairs.

How it's wired

`InMemoryChatMessageHistory` from `langchain_core`. The prompt template's `MessagesPlaceholder(variable_name="chat_history")` threads it through. `_trim_memory` trims FIFO.

Sidebar · Enable Conversation Memory · Memory Size

TOPIC 03 · DEMO

Toggle context on and off.

app/agent_app.py

Same model, same DB. Flip one knob at a time.

1 **Distinct values OFF**

Ask `"What activity type codes are in the database?"` — watch the agent guess.

2 **A question that needs the skill**

`"Year-over-year passenger growth from 2023 to 2024?"` — without the skill, transit gets counted, denominator's wrong.

3 **Memory on → follow-up**

`"Show me passengers in 2024."` then `"What about 2023?"` — agent resolves the reference. Toggle memory off; it can't.

TOPIC 03 · TAKEAWAY

Context is a data-engineering discipline. Curate it.

Three layers, one prompt.

`schema + distinct values` · `skills` · `memory`

Next → Topic 04 · what the agent is allowed to do.

TOPIC 04 · 10 MIN

SQL AI Agent guardrails.

Context tells the agent what it should do. Guardrails decide what it's allowed to do.

TOPIC 04 · THREE GUARDRAILS

Three layers between LLM output and the DB.

GUARDRAIL 01

Admin control — read-only

Only `Select`, `With`, `Show`, `Describe`. Everything else is blocked at the validator.

GUARDRAIL 02

SQL validation

`sqlglot` parses first. Multi-statement input is blocked. Unparseable strings are conservatively blocked.

GUARDRAIL 03

LIMIT enforcement

Every `SELECT` gets a `LIMIT`. Caps unbounded scans before they kill the workshop laptop.

TOPIC 04 · THE DEFAULTS THAT MATTER

ValidationConfig.

Eight lines of safety. Defined in `sql_ai_agent/sql_validator.py`.

```python
@dataclass
class ValidationConfig:
    read_only: bool = True
    max_limit: int = 10000
    enforce_limit: bool = True
    allowed_statements: list = field(
        default_factory=lambda: ["Select", "With", "Show", "Describe"]
    )
```

TOPIC 04 · LET'S TRY TO BREAK IT

Six prompts, six outcomes.

USER ASKS	LLM MIGHT GENERATE	VALIDATOR OUTCOME
Drop the air_traffic table.	`DROP TABLE air_traffic`	Blocked — read-only
Delete all rows where Year is 1999.	`DELETE FROM ...`	Blocked — read-only
Update Operating Airline to 'X'.	`UPDATE ... SET ...`	Blocked — read-only
Show all rows in the table.	`SELECT * FROM air_traffic`	Allowed; LIMIT 10 added
Show 2024 data; DROP TABLE ...	`SELECT ... ; DROP ...`	Blocked — multi-statement
Top 5 airlines by passenger count.	`SELECT ... LIMIT 5`	Allowed; LIMIT 5 kept

Safety beats correction.

When validation fails in read-only mode, `SqlAgent.ask_question` returns immediately. No retry. No fallback. Don't give the agent a second chance to ask for a destructive query.

TOPIC 04 · TAKEAWAY

Validate before you execute.

The validator is the only thing that knows the difference between "answer" and "modify".

`read-only` · `parse first` · `enforce LIMIT`

Next → Topic 05 · choosing the model.

TOPIC 05 · 15 MIN

Model performance & evaluation.

Without an eval, model selection collapses to vibes.

TOPIC 05 · WHY EVAL MATTERS

Define "working" before you pick.

The model card won't tell us how a model handles `air_traffic`. Pricing tier won't tell us either. Brand won't tell us at all.

What to measure

Success rate · latency · fix rate. Three numbers, three different questions.

Against what

A fixed test suite. The suite is the spec — pick the model after it exists.

How often

Every prompt change, every model bump. Drift is real.

TOPIC 05 · THE FRAMEWORK

sql_ai_agent.testing.

What it provides

`TestCase · TestRunner`
`get_test_cases · get_test_summary`

Each model gets **20 query-generation tests** + **5 debug-mechanism tests**. Twenty-five runs per model.

The suite (today)

`5 easy · 8 medium · 7 hard`

Eleven categories: basic · aggregation · filtering · comparison · ranking · time series · growth · percentage · market share · complex filtering · edge cases.

TOPIC 05 · THE FIELD

Eight models, two providers.

OpenAI

```
gpt-5.5
gpt-5.5-pro
gpt-5.4-mini
gpt-5.4-nano
gpt-4.1
```

Docker Model Runner (local)

```
ai/granite-4.0-h-micro
ai/llama3.2:latest
ai/gemma4:100k
```

Always include a cheap model and a local model in the comparison. The default isn't always the right choice.

TOPIC 05 · RESULTS · RUN 20260503_150027

The headline.

MODEL	PROVIDER	SUCCESS	AVG TIME (S)	FIX RATE
gpt-5.4-mini	**openai**	**90.00%**	**1.192**	**100%**
gpt-5.5	openai	90.00%	5.723	100%
gpt-4.1	openai	80.00%	0.825	100%
ai/granite-4.0-h-micro	docker	70.00%	2.828	80%
ai/gemma4:100k	docker	65.00%	1.956	100%
gpt-5.4-nano	openai	50.00%	3.376	100%
ai/llama3.2:latest	docker	40.00%	1.577	80%
gpt-5.5-pro	openai	0.00%	0.117	—

TOPIC 05 · THE CHARTS

Three views, three stories.

`success_rates_20260503_150027.html`

Quality bar

gpt-5.4-mini ties gpt-5.5 at 90%, ~5× faster.

`execution_times_20260503_150027.html`

Latency rank

gpt-4.1 fastest at 0.8s; gpt-5.5 slowest at 5.7s. Both clear 80%.

`difficulty_heatmap_20260503_150027.html`

Easy vs hard

100/30 ≠ 70/70. The heatmap is the only place that surfaces this.

gpt-5.4-mini ≈ gpt-5.5 at 90% success, at ~5× lower latency.

And: `gpt-5.5-pro` failed every test — a `404 not a chat model` config bug. Branding hid it. The eval surfaced it.

TOPIC 05 · THREE RULES FOR SHIPPING

For the next time you pick a model.

1 **Define the tests first.**
The test suite is the spec. Pick the model after it exists, not before.

2 **Always include a cheap and a local model.**
The default isn't always the right choice. Local models hit 65–70% here, with full data privacy.

3 **Re-run the eval whenever anything changes.**
Prompt template, schema, skill file, model version. Drift is real. Production regressions are not cheap.

TOPIC 05 · TAKEAWAY

Eval surfaces what branding hides.

Pick the cheapest model that meets your bar.

`success rate` · `latency` · `fix rate`

Next → Topic 06 · best practices we've earned.

TOPIC 06 · 8 MIN · RECAP

Best practices.

Five threads through everything we've covered.

TOPIC 06 · FIVE PRACTICES

When you go ship one of these.

1 **Context is a data-engineering discipline.**
Curating distinct values and the skill file is engineering work. Treat it like a pipeline you own.

2 **Skills compound.**
Write the domain knowledge down once, version it. Every new edge-case rule benefits every future query.

3 **Short-term and long-term memory serve different jobs.**
In-process chat history for follow-ups; DB-backed Q&A logs for cross-session learning.

4 **Measure, don't guess.**
Re-run the eval whenever you change the prompt, the schema, the skill file, or the model.

5 **Plan for the model to fail.**
Hosted models rate-limit, time out, return un-parseable output. Configure a fallback. A local model is a strong one.

TOPIC 06 · HONEST SCOPE

What's not in this workshop.

We kept the scope narrow on purpose.

Out of scope today

- tool-using agents (beyond SQL)
- multi-agent orchestration
- ReAct / planner-style architectures
- fine-tuning your own model

What carries over

Context, guardrails, evaluation, and fallback all transfer to those settings unchanged. The SQL agent is the warm-up.

CLOSING

Thanks for following along.

A SQL AI agent gets reliable when we treat it as a system, not a prompt.

`context` is engineered · `guardrails` are enforced · `models` are measured · `failure` has a fallback

Q&A · 10 min